Московский Авиационный Институт

(Национальный Исследовательский Университет)

Институт №8 “Компьютерные науки и прикладная математика”

Кафедра №806 “Вычислительная математика и программирование”

Лабораторная работа №1 по курсу

«Операционные системы»

Группа: М8О-210БВ-25

Студент: Пономарев Максим Сергеевич

Оценка: _____

Дата _____

Москва, 2026

Постановка задачи

Вариант 20. Родитель создаёт два дочерних процесса. Первые две строки ввода задают имена их выходных файлов. Следующие строки длиной не более 10 символов отправляются первому ребёнку через pipe1, остальные — второму через pipe2. Каждый ребёнок переворачивает полученные строки и записывает их в свой файл через стандартный вывод.

Общий метод и алгоритм решения

В parent.cpp находятся два явных вызова fork и две дочерние ветки. В первой ветке закрываются лишние концы каналов, open создаёт первый файл, dup2 подключает pipe1[0] к стандартному вводу и файл к стандартному выводу. execlp запускает ./child1. Во второй ветке аналогично запускается ./child2 с pipe2 и вторым файлом. Родитель закрывает оба читающих конца и распределяет строки по длине. После завершения ввода он закрывает оба пишущих конца и вызывает waitpid для каждого ребёнка.

Функции read_line, write_all и print_error записаны прямо в исходниках. read_line накапливает строку побайтовым чтением. write_all досылает остаток после частичной записи. При EINTR операции повторяются. signal(SIGPIPE, SIG_IGN) позволяет родителю обработать ошибку записи, если ребёнок закрыл канал. Существующие файлы защищены флагом O_EXCL. Сообщения об ошибках выводятся через write в стандартный поток ошибок. Разрешённый std::string используется для хранения строк.

Ожидается корректный UTF-8. Длина строки измеряется в кодовых точках без перевода строки. Байты продолжения символа UTF-8 находятся в диапазоне от 128 до 191. Родитель считает остальные байты. Ребёнок идёт с конца строки и копирует байты каждого символа в исходном порядке. Так сохраняется поддержка кириллицы. Графемные сочетания из нескольких кодовых точек отдельно не объединяются.

parent.cpp

```
#include <fcntl.h>
#include <signal.h>
#include <sys/wait.h>
#include <unistd.h>
#include <string>
#include <cerrno>
using namespace std;


bool write_all(int fd, const string& text)
{
    size_t offset = 0;
    while (offset < text.size())
    {
        ssize_t sent = write(fd, text.data() + offset, text.size() - offset);
        if (sent == -1)
        {
            if (errno == EINTR) continue;
            return false;
        }
        if (sent == 0) return false;
        offset += static_cast<size_t>(sent);
    }
    return true;
}


void print_error(const string& message)
{
    write_all(STDERR_FILENO, message + "\n");
}


// 1 — прочитана строка, 0 — конец ввода, -1 — ошибка.
int read_line(string& line)
{
    line.clear();
    char symbol;
    while (true)
    {
        ssize_t received = read(STDIN_FILENO, &symbol, 1);
        if (received == -1)
        {
            if (errno == EINTR) continue;
            return -1;
        }
        if (received == 0)
        {
            if (line.empty()) return 0;
            return 1;
        }
        if (symbol == '\n') return 1;
        line += symbol;
    }
}


bool wait_child(pid_t pid)
{
    int status = 0;
    pid_t result = waitpid(pid, &status, 0);
    while (result == -1 && errno == EINTR)
        result = waitpid(pid, &status, 0);
    if (result == -1)
    {
        print_error("waitpid failed");
        return false;
    }
    if (!WIFEXITED(status) || WEXITSTATUS(status) != 0)
    {
        print_error("Child finished with an error");
        return false;
    }
    return true;
}


int main()
{
    string filename1, filename2;
```

```
if (read_line(filename1) != 1 || read_line(filename2) != 1 ||
    filename1.empty() || filename2.empty())
{
    print_error("Enter two output filenames");
    return 1;
}
if (filename1 == filename2 || filename1.find('\0') != string::npos ||
    filename2.find('\0') != string::npos)
{
    print_error("Invalid output filenames");
    return 1;
}
if (signal(SIGPIPE, SIG_IGN) == SIG_ERR)
{
    print_error("signal failed");
    return 1;
}


int pipe1[2], pipe2[2];
if (pipe(pipe1) == -1)
{
    print_error("pipe1 failed");
    return 1;
}
if (pipe(pipe2) == -1)
{
    print_error("pipe2 failed");
    close(pipe1[0]);
    close(pipe1[1]);
    return 1;
}


pid_t pid1 = fork();
if (pid1 == -1)
{
    print_error("first fork failed");
    close(pipe1[0]);
    close(pipe1[1]);
    close(pipe2[0]);
    close(pipe2[1]);
    return 1;
}
if (pid1 == 0)
{
    close(pipe1[1]);
    close(pipe2[0]);
    close(pipe2[1]);
    int file_fd = open(filename1.c_str(), O_WRONLY | O_CREAT | O_EXCL, 0644);
    if (file_fd == -1)
    {
        print_error("Cannot create first output file");
        return 1;
    }
    if (dup2(pipe1[0], STDIN_FILENO) == -1 ||
        dup2(file_fd, STDOUT_FILENO) == -1)
    {
        print_error("first child: dup2 failed");
        return 1;
    }
    close(pipe1[0]);
    close(file_fd);
    execlp("./child1", "child1", static_cast<char*>(nullptr));
    print_error("Cannot execute ./child1");
    return 1;
}


pid_t pid2 = fork();
if (pid2 == -1)
{
    print_error("second fork failed");
    close(pipe1[0]);
    close(pipe1[1]);
    close(pipe2[0]);
    close(pipe2[1]);
    wait_child(pid1);
    return 1;
}
if (pid2 == 0)
{
    close(pipe2[1]);
```

```
        close(pipe1[0]);
        close(pipe1[1]);
        int file_fd = open(filename2.c_str(), O_WRONLY | O_CREAT | O_EXCL, 0644);
        if (file_fd == -1)
        {
            print_error("Cannot create second output file");
            return 1;
        }
        if (dup2(pipe2[0], STDIN_FILENO) == -1 ||
            dup2(file_fd, STDOUT_FILENO) == -1)
        {
            print_error("second child: dup2 failed");
            return 1;
        }
        close(pipe2[0]);
        close(file_fd);
        execlp("./child2", "child2", static_cast<char*>(nullptr));
        print_error("Cannot execute ./child2");
        return 1;
    }


    close(pipe1[0]);
    close(pipe2[0]);
    string line;
    bool success = true;
    while (true)
    {
        int state = read_line(line);
        if (state == 0) break;
        if (state == -1)
        {
            print_error("parent: read failed");
            success = false;
            break;
        }
        size_t length = 0;
        for (size_t i = 0; i < line.size(); ++i)
        {
            unsigned char byte = static_cast<unsigned char>(line[i]);
            if (byte < 128 || byte >= 192) ++length;
        }
        int target = pipe1[1];
        if (length > 10) target = pipe2[1];
        line += '\n';
        if (!write_all(target, line))
        {
            print_error("parent: write failed");
            success = false;
            break;
        }
    }
    close(pipe1[1]);
    close(pipe2[1]);
    if (!wait_child(pid1)) success = false;
    if (!wait_child(pid2)) success = false;
    if (!success) return 1;
    return 0;
}
```

child1.cpp

```
// Дочерняя программа 1.
#include <unistd.h>
#include <string>
#include <cerrno>
using namespace std;


bool write_all(int fd, const string& text)
{
    size_t offset = 0;
    while (offset < text.size())
    {
        ssize_t sent = write(fd, text.data() + offset, text.size() - offset);
        if (sent == -1)
        {
            if (errno == EINTR) continue;
            return false;
        }
        if (sent == 0) return false;
        offset += static_cast<size_t>(sent);
    }
    return true;
}


void print_error(const string& message)
{
    write_all(STDERR_FILENO, message + "\n");
}


// 1 — прочитана строка, 0 — конец ввода, -1 — ошибка.
int read_line(string& line)
{
    line.clear();
    char symbol;
    while (true)
    {
        ssize_t received = read(STDIN_FILENO, &symbol, 1);
        if (received == -1)
        {
            if (errno == EINTR) continue;
            return -1;
        }
        if (received == 0)
        {
            if (line.empty()) return 0;
            return 1;
        }
        if (symbol == '\n') return 1;
        line += symbol;
    }
}


// Переворачиваем символы UTF-8, сохраняя порядок байтов внутри символа.
bool send_reversed(string& line)
{
    string result;
    size_t end = line.size();
    while (end > 0)
    {
        size_t start = end - 1;
        while (start > 0)
        {
            unsigned char byte = static_cast<unsigned char>(line[start]);
            if (byte < 128 || byte >= 192) break;
            --start;
        }
        for (size_t i = start; i < end; ++i)
            result += line[i];
        end = start;
    }
    result += '\n';
    if (!write_all(STDOUT_FILENO, result)) return false;
    line.clear();
    return true;
}
```

```
int main()
{
    string line;
    while (true)
    {
        int state = read_line(line);
        if (state == 0) break;
        if (state == -1)
        {
            print_error("read failed");
            return 1;
        }
        if (!send_reversed(line))
        {
            print_error("write failed");
            return 1;
        }
    }
    return 0;
}
```

child2.cpp

```
// Дочерняя программа 2.
#include <unistd.h>
#include <string>
#include <cerrno>
using namespace std;


bool write_all(int fd, const string& text)
{
    size_t offset = 0;
    while (offset < text.size())
    {
        ssize_t sent = write(fd, text.data() + offset, text.size() - offset);
        if (sent == -1)
        {
            if (errno == EINTR) continue;
            return false;
        }
        if (sent == 0) return false;
        offset += static_cast<size_t>(sent);
    }
    return true;
}


void print_error(const string& message)
{
    write_all(STDERR_FILENO, message + "\n");
}


// 1 — прочитана строка, 0 — конец ввода, -1 — ошибка.
int read_line(string& line)
{
    line.clear();
    char symbol;
    while (true)
    {
        ssize_t received = read(STDIN_FILENO, &symbol, 1);
        if (received == -1)
        {
            if (errno == EINTR) continue;
            return -1;
        }
        if (received == 0)
        {
            if (line.empty()) return 0;
            return 1;
        }
        if (symbol == '\n') return 1;
        line += symbol;
    }
}


// Переворачиваем символы UTF-8, сохраняя порядок байтов внутри символа.
bool send_reversed(string& line)
{
    string result;
    size_t end = line.size();
    while (end > 0)
    {
        size_t start = end - 1;
        while (start > 0)
        {
            unsigned char byte = static_cast<unsigned char>(line[start]);
            if (byte < 128 || byte >= 192) break;
            --start;
        }
        for (size_t i = start; i < end; ++i)
            result += line[i];
        end = start;
    }
    result += '\n';
    if (!write_all(STDOUT_FILENO, result)) return false;
    line.clear();
    return true;
}
```

```
int main()
{
    string line;
    while (true)
    {
        int state = read_line(line);
        if (state == 0) break;
        if (state == -1)
        {
            print_error("read failed");
            return 1;
        }
        if (!send_reversed(line))
        {
            print_error("write failed");
            return 1;
        }
    }
    return 0;
}
```

Протокол работы программы

```
g++ -Wall -Wextra -Wpedantic child1.cpp -o child1
g++ -Wall -Wextra -Wpedantic child2.cpp -o child2
g++ -Wall -Wextra -Wpedantic parent.cpp -o parent
./parent
После двух новых имён файлов вводились строки:
1234567890
abcdefghijk


Привет
абвгдежзийк
last


Первый файл:
0987654321


тевирП
tsal


Второй файл:
kjihgfedcba
кйизжедгвба
```

Вывод

Сборка трёх программ в WSL с -Wall -Wextra -Wpedantic прошла без предупреждений. Автоматическое сравнение результатов подтвердило распределение на границе 10/11 символов, обработку пустой строки, кириллицы и последней строки без перевода. Дополнительно проверены строки до 100000 байтов, 400 коротких строк, пустой набор данных и неверные имена файлов. При повторном использовании имён получена ошибка, прежние файлы сохранили своё содержимое.